CE-1103 · Lección 04

Ordenamiento y Búsqueda

Encontrar y ordenar datos de forma eficiente

Busqueda.java

```java
int[] a = {5, 3, 8, 6, 2};
selectionSort(a);
// a: 2, 3, 5, 6, 8
int i = binarySearch(a, 6);
// i == 3
```

Instituto Tecnológico de Costa Rica · Ingeniería en Computadores

// introducción

¿Qué aprenderemos hoy?

1. Buscar con búsqueda lineal, binaria y hash
2. Ordenar con selección, burbuja e inserción
3. Ordenar con quick sort, Shellsort y merge sort
4. Ordenar con radix sort, dígito por dígito
5. Medir el costo con la notación O y comparar los algoritmos

// costo de un algoritmo

Medir el costo: contar operaciones

¿Cómo decidimos cuál de dos algoritmos es mejor?

Segundos: no sirven

Dependen de la computadora, el compilador, el lenguaje y otros programas.

Cambian de equipo en equipo

Contar operaciones

Se cuenta cuántas veces se repite la operación central, según el tamaño n.

Aquí: las comparaciones

Mejor y peor caso

Depende de los datos. Lineal con n = 9: 1 comparación en el mejor caso, 9 en el peor.

En promedio, ≈ n/2

En esta lección la operación central será **comparar** dos valores.

// costo de un algoritmo

Notación O: el término que más crece

Orden	n = 10	n = 100	n = 1 000	En esta lección
O(1)	1	1	1	Hash (promedio)
O(log n)	≈ 3,3	≈ 6,6	≈ 10	Búsqueda binaria
O(n)	10	100	1 000	Búsqueda lineal
O(n log n)	≈ 33	≈ 664	≈ 9 966	Quick sort, merge sort
O(n²)	100	10 000	1 000 000	Selección, burbuja, inserción

Notación O: se conserva solo el término que más crece. $2n^3 + 3n^2 + \log n + 333$ es **O(n³)**.

Ojo: con n = 10 000, el término $2n^3$ ya aporta más del 99,9 % del total.

// búsqueda

Tres formas de buscar

Buscar un elemento dentro de una colección; en el curso, un arreglo:

1

Búsqueda lineal

Recorre el arreglo de principio a fin, comparando cada elemento con el valor buscado.

Cualquier arreglo

2

Búsqueda binaria

Compara con el elemento del medio y descarta la mitad donde el valor no puede estar.

Arreglo ordenado

3

Búsqueda hash

Una función de hash convierte la llave en el índice donde se guarda el valor.

Sin recorrer el arreglo

// búsqueda lineal

Búsqueda lineal: buscar el 7

Comparando Ya descartado Encontrado

	0	1	2	3	4	5	6	7	8	
i = 0	**5**	3	8	6	2	7	1	4	9	5 ≠ 7: siga
i = 1	5	**3**	8	6	2	7	1	4	9	3 ≠ 7: siga
i = 2	5	3	**8**	6	2	7	1	4	9	8 ≠ 7: siga
i = 3	5	3	8	**6**	2	7	1	4	9	6 ≠ 7: siga
i = 4	5	3	8	6	**2**	7	1	4	9	2 ≠ 7: siga
i = 5	5	3	8	6	2	**7**	1	4	9	**7 = 7: encontrado en la posición 5**

// búsqueda binaria

Búsqueda binaria: buscar el 9



// búsqueda binaria

Búsqueda lineal y binaria en Java

Lineal · O(n)

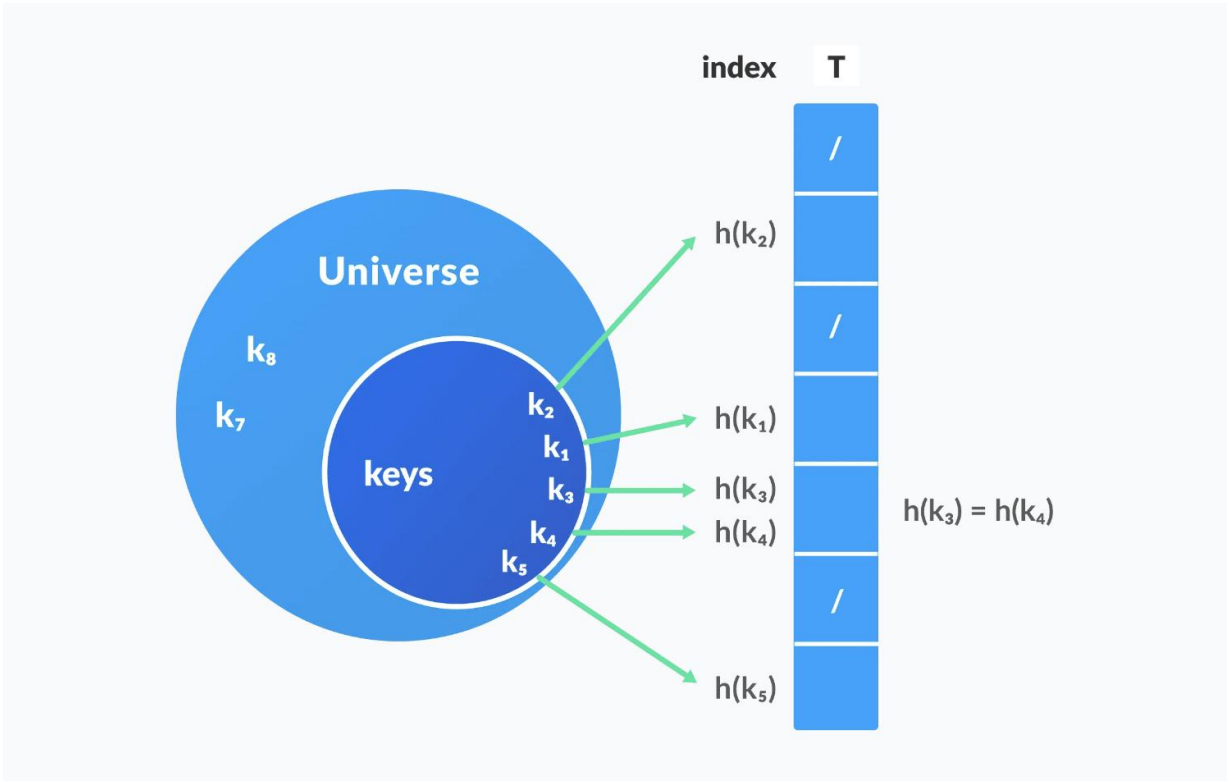
```java
static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}
```

Binaria (iterativa) · O(log n)

```java
static int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

Ambas devuelven la **posición** del valor o **-1** si no está. La binaria solo funciona con el arreglo **ordenado**.

Tabla hash: de la llave al índice



La llave k entra a la función h(k), que produce el índice i donde se guarda el valor.

Ejemplo: cédulas con h(k) = k % 10

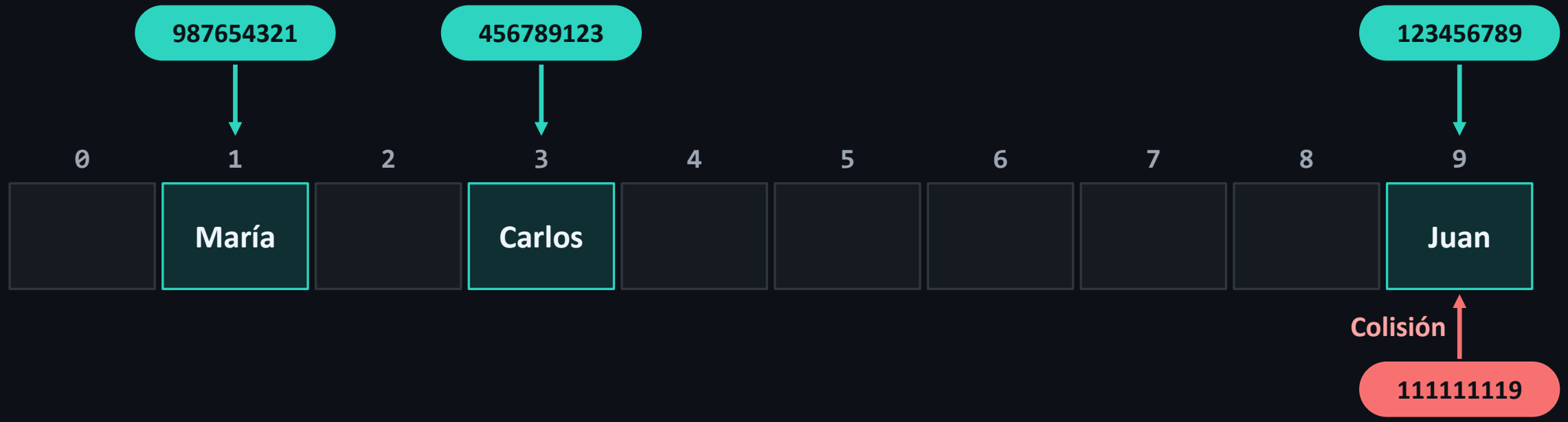
```
h(cédula) = cédula % 10
```



Para buscar una cédula se calcula `h(cédula)` y se va **directo** al índice: no hay que recorrer el arreglo.

Colisión: dos llaves, el mismo índice

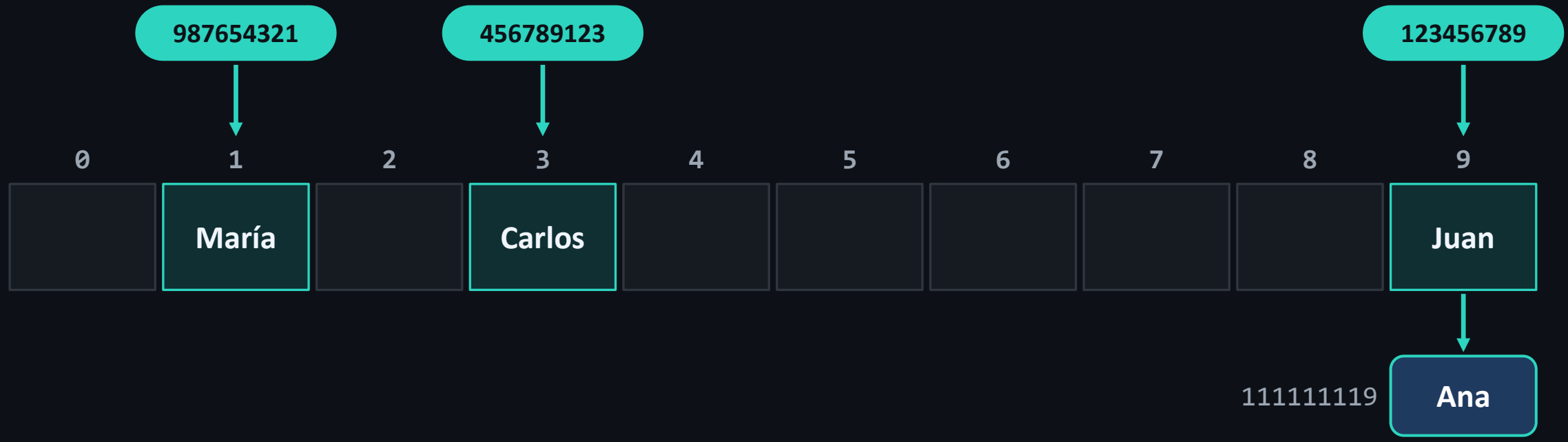
```
h(cédula) = cédula % 10
```



Colisión: `h(111111119) = 9`, pero el índice 9 ya está ocupado por Juan. ¿Dónde se guarda?

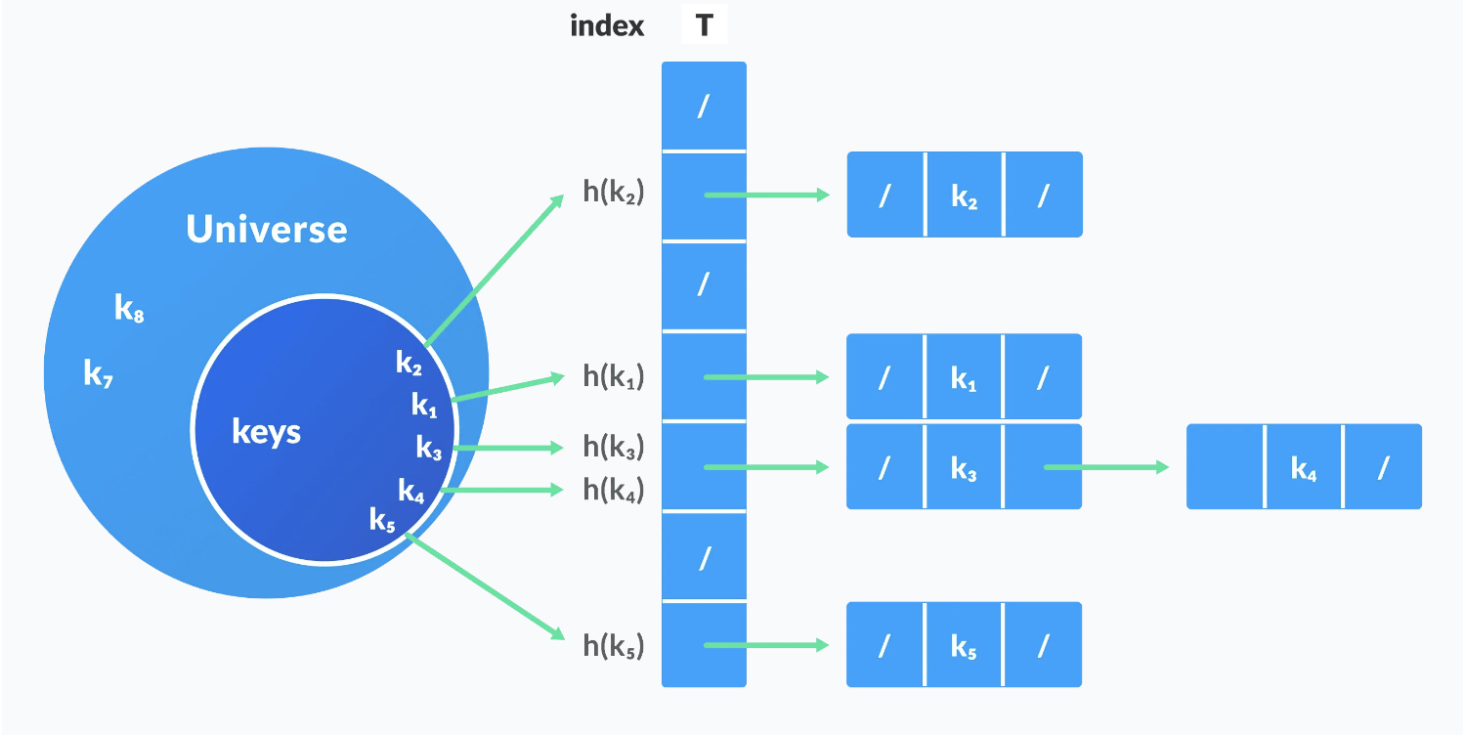
Separate chaining con las cédulas

```
h(cédula) = cédula % 10
```



Cada índice guarda una **lista enlazada**: para buscar 111111119 se va al índice 9 y se recorre su lista.

Separate chaining: la lista de cada índice



Los elementos que colisionan (k3 y k4) se encadenan en una lista enlazada.

Hash: la función decide los buckets

```
h(cédula) = cédula % 13
```



Con % 10: solo cuenta el último dígito; 123456789 y 111111119 chocaban en el 9.

Con % 13: el divisor define los buckets y suele ser un **primo**; las cuatro cédulas caen en 1, 2, 4 y 11.

Otras funciones de hash

Método	Idea	Ejemplo
Módulo	Residuo de dividir la llave entre un primo	12345678 % 13 = 7
Cuadrado medio	Elevar al cuadrado y tomar los dígitos centrales	$123^2 = 15129 \rightarrow 51$
Truncamiento	Ignorar parte de la llave y usar el resto	12345678 → dígitos 1, 3 y 8 → 138
Plegado	Dividir la llave en partes y sumarlas	123 + 456 + 78 = 657

Colisión: con truncamiento, 71140205 y 73162135 dan el mismo índice, 715.

Una buena función reparte las llaves por **toda** la tabla; una mala deja casi todos los buckets vacíos.

Comparando los algoritmos de búsqueda

Algoritmo	¿Requiere orden?	Costo promedio	Peor caso
Lineal	No	O(n)	O(n)
Binaria	Sí	O(log n)	O(log n)
Hash	No	O(1)	O(n): todas las llaves colisionan

Con 1 000 000 de elementos: la lineal necesita hasta 1 000 000 de comparaciones; la binaria, hasta 20.

Ojo: hash tiene el mejor rendimiento promedio, pero depende de la función de hash y de las colisiones.

Los algoritmos de ordenamiento

Simples

Selección, burbuja e inserción. Fáciles de implementar; $O(n^2)$ en el peor caso.

Recursivos

Quick sort y merge sort: dividen el arreglo y ordenan cada parte.

Incrementos

Shellsort: inserción sobre sublistas separadas por un gap cada vez menor.

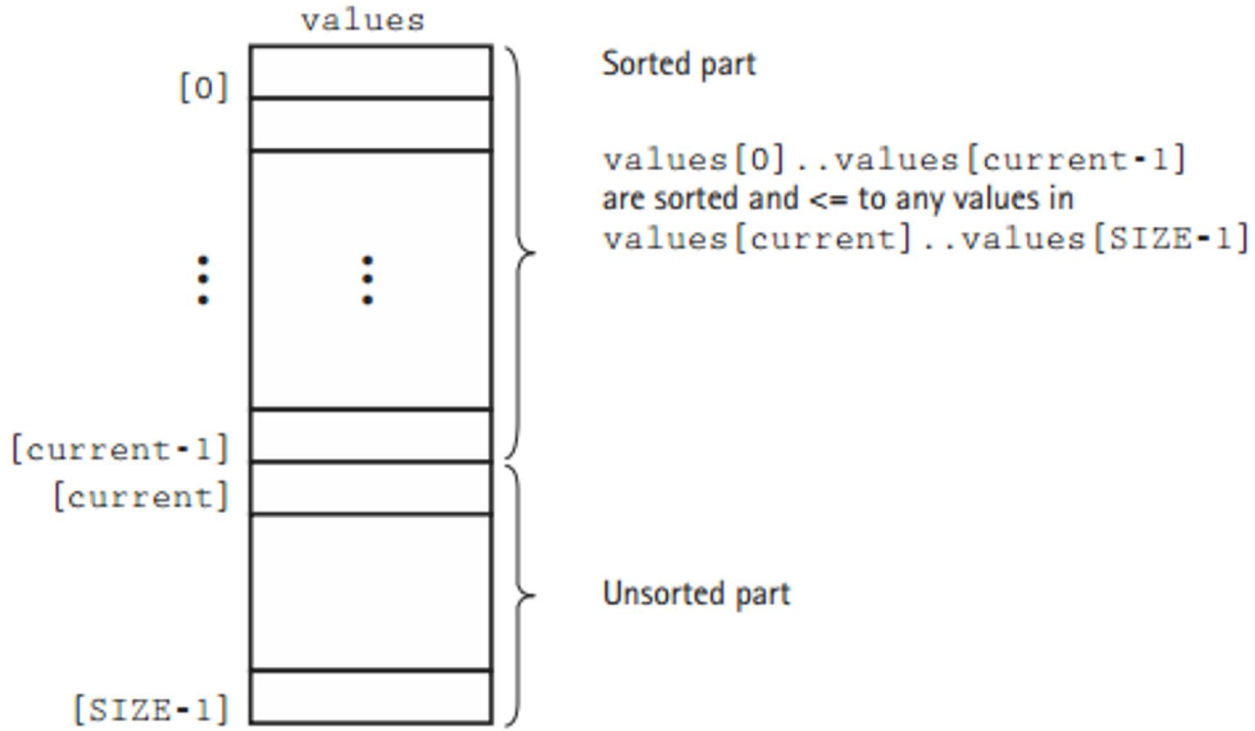
Por dígitos

Radix sort: procesa dígito por dígito, sin comparar elementos completos.

> Hay muchos algoritmos porque cada uno tiene **diferentes características de rendimiento**.

// ordenamiento por selección

Selección: parte ordenada y parte desordenada



Cada pasada mueve el menor de la parte desordenada al final de la parte ordenada.

// ordenamiento por selección

Selección: cuatro pasadas



// ordenamiento por selección

Selección en Java

```java
public static void selectionSort(int[] arr) {
    for (int i = 0; i < arr.length - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < arr.length; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        int temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}
```

Ventajas

Fácil de entender e implementar. No necesita memoria adicional significativa. Funciona bien con listas pequeñas.

Desventajas

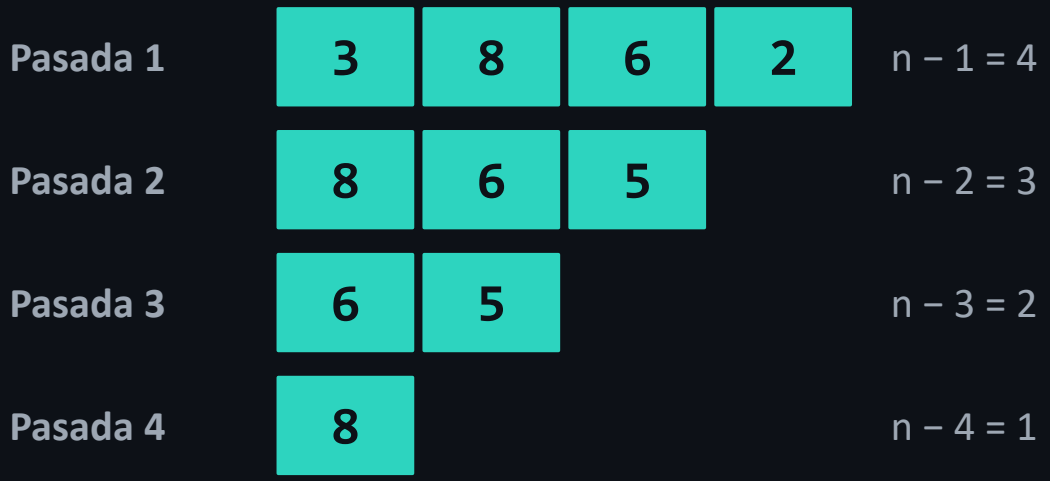
$O(n^2)$: ineficiente con muchos datos. Siempre hace las mismas comparaciones, sin importar el orden.

El ciclo interno busca el mínimo; el intercambio se hace **una sola vez** por pasada.

// ordenamiento por selección

Selección: ¿cuántas comparaciones?

Cada celda es una comparación `arr[j] < arr[minIndex]`:



Total: 4 + 3 + 2 + 1 = 10 = 5 · 4 / 2

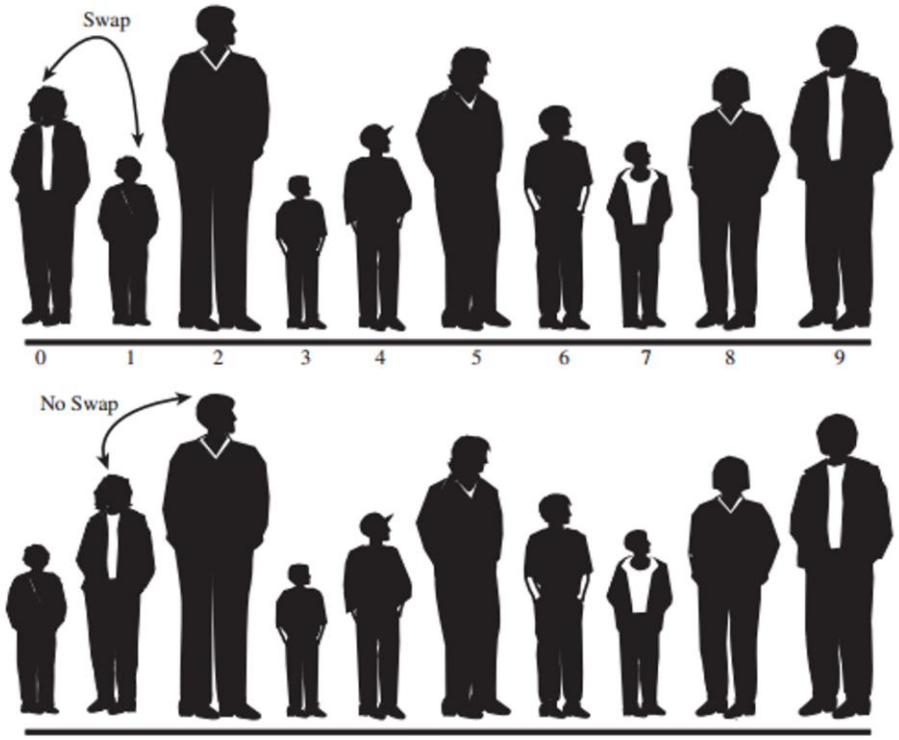
Para n elementos: n(n − 1)/2

n	Comparaciones
5	10
10	45
100	4 950
1 000	499 500
10 000	49 995 000

O(n²): si n se multiplica por 10, las comparaciones se multiplican por ≈ 100. Burbuja hace exactamente las mismas.

// ordenamiento de burbuja

Burbuja: comparar elementos vecinos



Si el primero es mayor que el segundo se intercambian (Swap); si no, no se hace nada (No Swap).

// ordenamiento de burbuja

Burbuja: la primera pasada

Comparación sin cambio · Intercambio · En su lugar

Inicio	5	3	8	6	2	Se compara cada par de vecinos
j = 0	3	5	8	6	2	5 > 3: intercambio
j = 1	3	5	8	6	2	5 < 8: no hay cambio
j = 2	3	5	6	8	2	8 > 6: intercambio
j = 3	3	5	6	2	8	8 > 2: intercambio
Fin de la pasada 1	3	5	6	2	8	**El mayor (8) quedó al final**

// ordenamiento de burbuja

Burbuja en Java

```java
public static void bubbleSort(int[] arr) {
    for (int i = 0; i < arr.length - 1; i++) {
        for (int j = 0; j < arr.length - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Ventajas

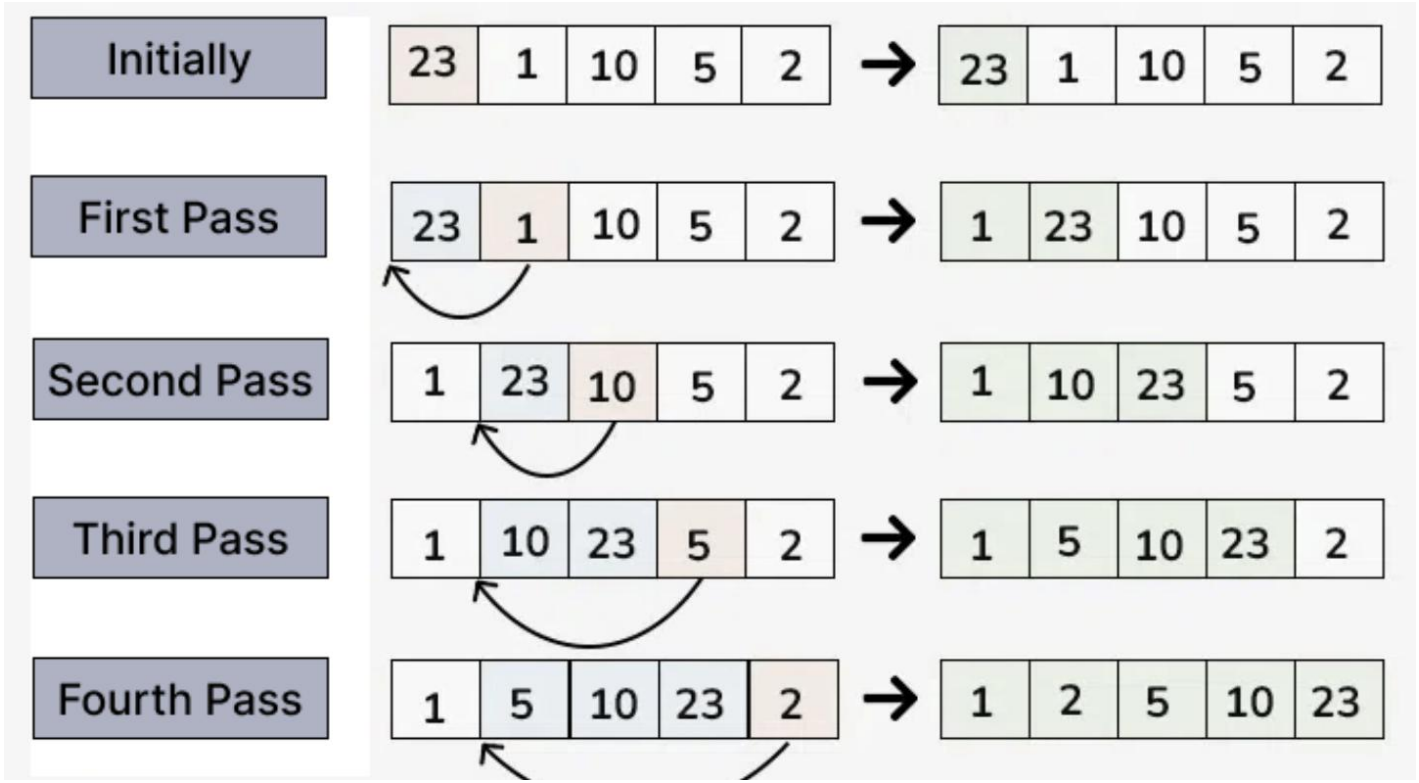
Fácil de entender e implementar, igual que selección.

Desventajas

O(n^2): ineficiente para grandes conjuntos de datos.

> Tras cada pasada, el mayor de los elementos restantes queda al final y ya no se vuelve a comparar.

Inserción: insertar en la parte ordenada



Cada elemento se inserta, de atrás hacia adelante, en su lugar dentro de la lista ordenada.

// ordenamiento por inserción

Inserción en Java

```java
public static void insertionSort(int[] arr) {
    for (int i = 1; i < arr.length; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

Mejor caso

Listas pequeñas o parcialmente ordenadas.

O(n)

Peor caso

Degenera a una complejidad cuadrática.

$O(n^2)$

Cada elemento (key) se **inserta** desplazando a la derecha los mayores de la parte ordenada.

// quick sort

Quick sort: la idea

1

Pivote

Se elige un elemento como pivote; usualmente el central.

2

Partición

Los menores quedan a la izquierda del pivote y los mayores a la derecha.

3

Recursión

Se repite en los subarreglos izquierdo y derecho del pivote.

4

Caso base

Un subarreglo de un solo elemento ya está ordenado.

Quick sort (ordenamiento rápido) es un algoritmo de ordenamiento **eficiente y rápido**.

Quick sort: una partición



Ahora se repite el proceso con `[3, 1, 2, 0]` (izquierda) y con `[10, 8]` (derecha).

// quick sort

Quick sort: la recursión

Subarreglo por ordenar En su posición final



Caso base: un subarreglo de un solo elemento ya está ordenado, y la recursión termina.

// quick sort

Quick sort en Java: la recursión

```java
static void quickSort(int[] array, int left, int right) {
    if (left < right) {
        int pivotIndex = partition(array, left, right);
        quickSort(array, left, pivotIndex - 1);
        quickSort(array, pivotIndex + 1, right);
    }
}
```

```java
static void swap(int[] array, int a, int b) {
    int temp = array[a];
    array[a] = array[b];
    array[b] = temp;
}
```

Recursión

Se llama a sí mismo con la parte izquierda y la derecha del pivote.

Costo promedio

Eficiente en la práctica.

O(n log n)

// quick sort

Quick sort en Java: la partición

```java
static int partition(int[] array, int left, int right) {
    int pivotIndex = left + (right - left) / 2;
    int pivot = array[pivotIndex];
    swap(array, pivotIndex, right);
    int partitionIndex = left;
    for (int i = left; i < right; i++) {
        if (array[i] <= pivot) {
            swap(array, i, partitionIndex);
            partitionIndex++;
        }
    }
    swap(array, partitionIndex, right);
    return partitionIndex;
}
```

Los tres pasos

1. Pivote al final: el central se intercambia con el último.
2. Menores al frente: los ≤ pivote pasan a la izquierda; `partitionIndex` cuenta cuántos hay.
3. Pivote a su lugar: se intercambia con `partitionIndex`.

// shellsort

Shellsort: inserción con incrementos

Propuesto por Donald Shell en 1959:

1

Incrementos (gaps)

Ordena elementos separados por un gap; el gap se reduce hasta llegar a 1.

2

Basado en inserción

Aplica insertion sort a sublistas pequeñas y luego a sublistas cada vez más grandes.

3

Mejor caso

Aprovecha que la inserción es rápida con datos casi ordenados.

Gaps: cualquier secuencia decreciente que termine en **1**. Ejemplo con 9 elementos: **5, 3, 1**.

// shellsort

Shellsort: gaps 5, 3 y 1

Cambió de lugar Ordenado

	0	1	2	3	4	5	6	7	8	
Inicio	5	3	8	6	2	7	1	4	9	Arreglo de 9 elementos
gap = 5	5	1	4	6	2	7	3	8	9	Se intercambian 3 con 1 y 8 con 4
gap = 3	3	1	4	5	2	7	6	8	9	La sublista (5, 6, 3) queda (3, 5, 6)
gap = 1	1	2	3	4	5	6	7	8	9	**Inserción normal: ¡ordenado!**

Cada pasada deja el arreglo más cerca del orden final; la última (gap = 1) es una inserción normal.

// shellsort

Shellsort en Java

```java
static void shellSort(int[] arr) {
    int[] gaps = {5, 3, 1};
    for (int gap : gaps) {
        for (int i = gap; i < arr.length; i++) {
            int key = arr[i];
            int j = i;
            while (j >= gap && arr[j - gap] > key) {
                arr[j] = arr[j - gap];
                j -= gap;
            }
            arr[j] = key;
        }
    }
}
```

Con gap = 1

Es exactamente el insertion sort de antes.

Gaps

El ejemplo usa {5, 3, 1}. Knuth propone 1, 4, 13, 40… (h = 3h + 1). El último debe ser 1.

// merge sort

Merge sort: dividir y conquistar

1

Dividir

Parte el arreglo en dos mitades.

2

Ordenar

Ordena cada mitad de forma recursiva.

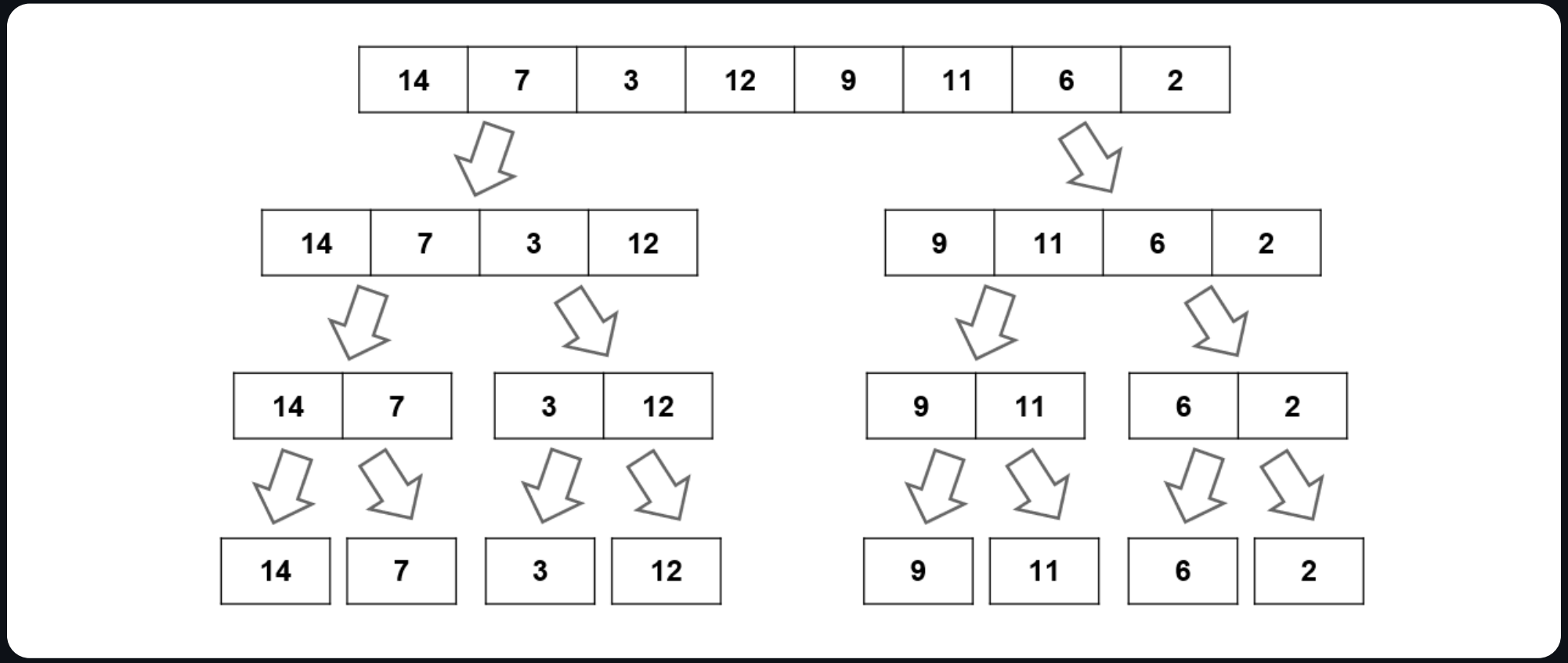
3

Combinar

Mezcla las dos mitades ya ordenadas en un solo arreglo ordenado.

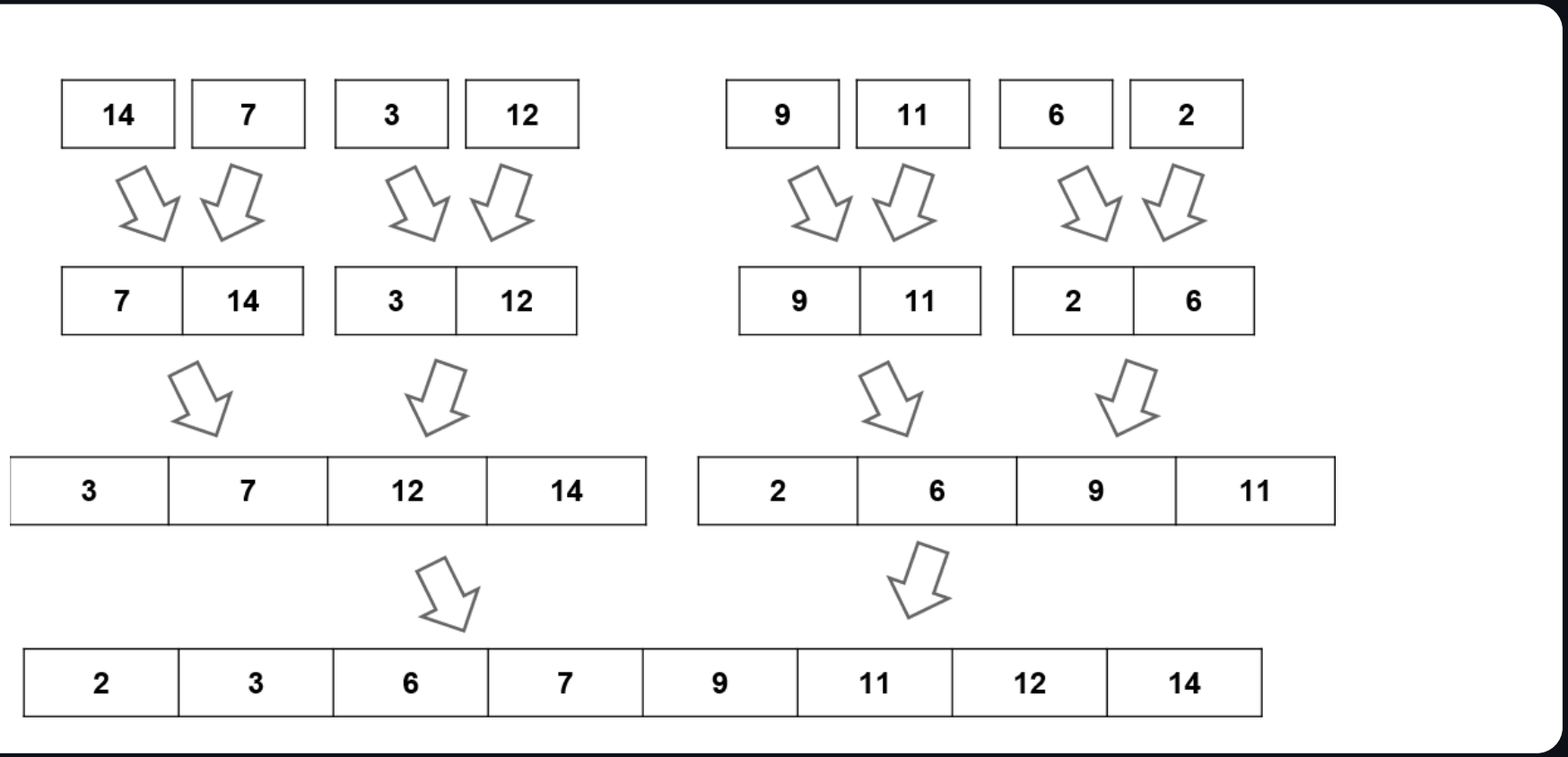
Ordenamiento por mezcla (merge sort): sigue el paradigma de **dividir y conquistar**.

Merge sort: fase de división



El arreglo se divide en mitades hasta quedar con elementos individuales.

Merge sort: fase de unión



Las mitades ordenadas se mezclan, de abajo hacia arriba, hasta formar el arreglo ordenado.

Merge sort: ¿por qué O(n log n)?

Se mezcla en este nivel Caso base: 1 elemento



3 niveles con mezcla ($\log_2 8$) × **8** elementos por nivel = 24 ≈ n $\log_2$ n. Con n = 1 024: 10 240 frente a 1 048 576 de n^2.

// merge sort

Merge sort en Java

```java
void mergeSort(int[] a, int low, int high) {
    if (low < high) {
        int mid = low + (high - low) / 2;
        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
        merge(a, low, high, mid);
    }
}
```

```java
void merge(int[] a, int low, int high, int mid) {
    int[] c = new int[high - low + 1];
    int i = low, j = mid + 1, k = 0;
    while (i <= mid && j <= high) {
        if (a[i] <= a[j]) c[k++] = a[i++];
        else c[k++] = a[j++];
    }
    while (i <= mid) c[k++] = a[i++];
    while (j <= high) c[k++] = a[j++];
    for (i = 0; i < k; i++) a[low + i] = c[i];
}
```

Costo

Siempre O(n log n). El arreglo auxiliar `c` solo tiene el tamaño del rango que se mezcla.

// radix sort

Radix sort: ordenar por dígitos

1

Dígito por dígito

No compara números completos: ordena por unidades, luego decenas, luego centenas…

2

Radix = raíz

Es la base del sistema numérico: 10 en decimal, 2 en binario.

3

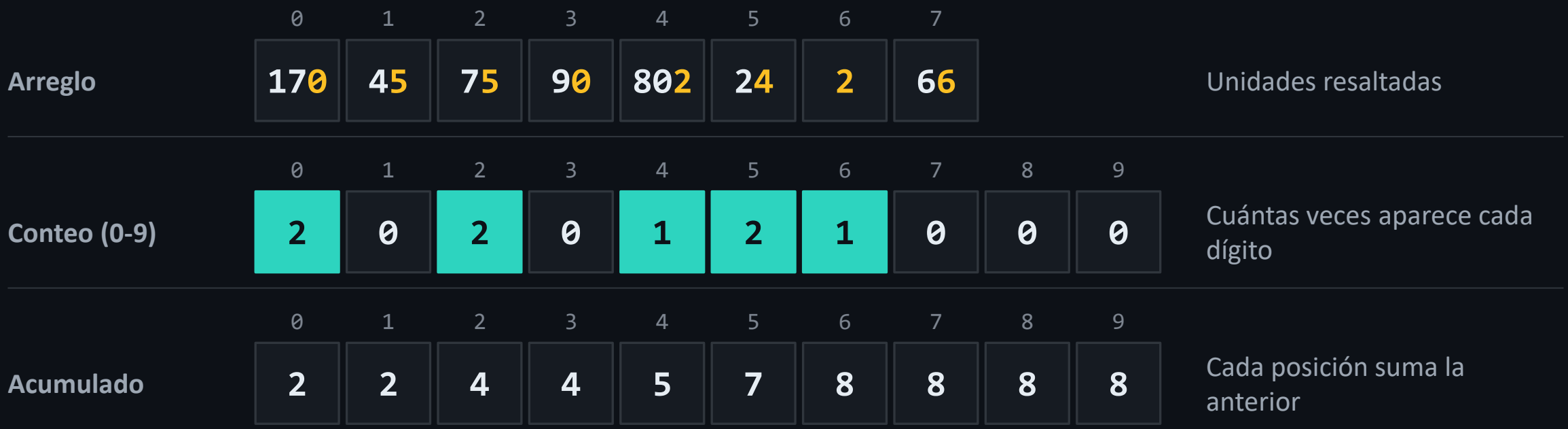
Arreglo de conteo

Una posición por cada dígito posible (0 a 9); cuenta cuántas veces aparece cada uno.

Ejemplo: `[170, 45, 75, 90, 802, 24, 2, 66]`

// radix sort

Radix sort: contar los dígitos de las unidades



El arreglo acumulado indica dónde termina cada dígito en el arreglo ordenado.

// radix sort

Radix sort: tres pasadas

Original	170	045	075	090	802	024	002	066	Con ceros a la izquierda
Por unidades	170	090	802	002	024	045	075	066	Unidades: 0, 0, 2, 2, 4, 5, 5, 6
Por decenas	802	002	024	045	066	170	075	090	Decenas: 0, 0, 2, 4, 6, 7, 7, 9
Por centenas	002	024	045	066	075	090	170	802	**Centenas: ordenado**

> Cada pasada debe ser **estable**: los dígitos iguales conservan el orden de la pasada anterior.

// radix sort

Radix sort en Java

```
static void radixSort(int[] arr) {
    int max = arr[0];
    for (int v : arr) max = Math.max(max, v);
    for (int exp = 1; max / exp > 0; exp *= 10)
        countSort(arr, exp);
}
```

Estable

Se recorre de atrás hacia adelante para que los dígitos iguales conserven su orden.

```
static void countSort(int[] arr, int exp) {
    int[] output = new int[arr.length];
    int[] count = new int[10];
    for (int v : arr) count[(v / exp) % 10]++;
    for (int d = 1; d < 10; d++)
        count[d] += count[d - 1];
    for (int i = arr.length - 1; i >= 0; i--) {
        int d = (arr[i] / exp) % 10;
        output[count[d] - 1] = arr[i];
        count[d]--;
    }
    for (int i = 0; i < arr.length; i++)
        arr[i] = output[i];
}
```

Ojo: solo funciona con enteros ≥ 0. Costo: **O(d · (n + b))**, con d dígitos y base b = 10.

// comparación

Complejidad: simples y Shellsort

Algoritmo	Mejor	Promedio	Peor	Estable	Memoria extra
Selección	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	O(1)
Burbuja	$O(n^2)$	$O(n^2)$	$O(n^2)$	Sí	O(1)
Inserción	O(n)	$O(n^2)$	$O(n^2)$	Sí	O(1)
Shellsort	O(n log n)	según los gaps	$O(n^2)$	No	O(1)

Estable: los elementos iguales conservan su orden relativo.

Burbuja: con una bandera «sin intercambios», su mejor caso sería O(n).

// comparación

Complejidad: quick, merge y radix

Algoritmo	Mejor	Promedio	Peor	Estable	Memoria extra
Quick sort	O(n log n)	O(n log n)	$O(n^2)$	No	O(log n) prom.
Merge sort	O(n log n)	O(n log n)	O(n log n)	Sí	O(n)
Radix sort	O(d·(n+b))	O(d·(n+b))	O(d·(n+b))	Sí	O(n+b)

Radix: d = número de dígitos; b = base (10 en decimal).

Quick sort: con pivotes muy desbalanceados el tiempo llega a $O(n^2)$ y la pila de llamadas, a O(n).

Merge sort: siempre O(n log n), a cambio de memoria extra.

// práctica

Ahora usted

Resuelva cada caso con lo aprendido:

A

Elegir una búsqueda

Un arreglo de 1 000 000 de elementos **sin ordenar**: ¿qué algoritmo usaría? ¿Y si estuviera ordenado?

B

Traza de selección

Aplique selección a `[4, 2, 7, 1]`. Escriba el arreglo al terminar cada pasada.

C

Colisión en un hash

Con `h(k) = k % 10`, ¿en qué índice se guardan las cédulas 111111115 y 222222225? ¿Qué ocurre?

Pista: en A compare cuánto cuesta cada algoritmo; en C recuerde cómo se resuelven las colisiones.

// cierre

Resumen

- **Costo:** se cuentan comparaciones; la notación **O** es el término que más crece.
- **Búsqueda:** lineal, binaria (arreglo ordenado) y **hash** con separate chaining.
- **Selección, burbuja e inserción:** simples, $O(n^2)$.
- **Quick sort** y **merge sort:** dividir y conquistar; O(n log n) en promedio.
- **Shellsort:** inserción con incrementos (gaps) cada vez más pequeños.
- **Radix sort:** ordena dígito por dígito con un arreglo de conteo.
- **Elegir:** según el rendimiento, la memoria extra y la estabilidad.

Recursos adicionales

- **Video introductorio a los algoritmos de búsqueda**
  youtube.com/watch?v=WaNLJf8xzC4
- **Quick sort: visualización interactiva**
  me.dt.in.th/page/Quicksort/
- **Quick sort: explicación y ejemplos**
  geeksforgeeks.org/quick-sort/
- **Visualización de algoritmos de ordenamiento por comparación**
  cs.usfca.edu/~galles/visualization/ComparisonSort.html